

Choisir et configurer le service d'IA

Call To Adventure · Bloc 2 · E2 : 15 minutes

BESOIN

Un maître du jeu narratif en français et en anglais, relié à un état de jeu.

DÉMARCHE

Veille, critères de sélection, architecture et configuration.

POSITIONNEMENT

Intégration de services préexistants. Pas d'entraînement d'un modèle de génération dans ce projet.

De la demande fonctionnelle à une solution intégrable.

Raconter, proposer, agir : trois contrats

L'IA intervient dans un moteur dont les règles restent côté serveur.

NARRATION

État + action + contexte
documentaire → suite de
l'aventure en FR ou EN.

CHOIX

Exactement 3 actions distinctes.
6 mots maximum par action.
Structure validée par Pydantic.

DÉCISIONS

Objectifs et salle : sorties typées.
Combat, soin et dégâts : outils
autorisés et règles déterministes.

Une sortie grammaticalement correcte peut rester incorrecte pour le jeu.

Définir la réussite avant de comparer

Les valeurs suivantes sont des objectifs documentés, pas des scores mesurés.

QUALITÉ

100 % de sorties structurées valides.
Au moins 90 % de réussite sur le jeu de cas.
Relecture humaine FR / EN.

SERVICE

Objectif p95 inférieur à 30 s.
Disponibilité logique visée : 99 %.
Échec contrôlé sans perte d'état.

CONTRAINTES

Budget mensuel à renseigner.
Conditions de traitement à vérifier.
Matériel local pour les embeddings.

Séparer validité du format, qualité de la réponse et disponibilité du service.

Une veille à rattacher aux décisions

Trame proposée ; les preuves de veille ne figurent pas dans le dépôt consulté.

THÈMES

Sorties structurées et outils.
RAG et évaluation.
Sécurité, données personnelles,
accessibilité et sobriété.

MÉTHODE PROPOSÉE

1 h hebdomadaire.
Agrégateur RSS + journal daté.
Auteur, date et fiabilité.
Recoupement des informations.

PREUVE ATTENDUE

Une synthèse accessible.
Une recommandation argumentée.
Une décision liée au projet.
Une trace de partage effective.

À compléter avec votre veille réelle ; aucune synthèse diffusée n'est inventée.

Comparer sur un protocole identique

Le benchmark réel de plusieurs modèles reste à réaliser.

MODÈLE HÉBERGÉ ACTUEL

Intégré dans le code.
Structure, langue, latence et coût à mesurer.
Choix encore provisoire.

SECOND SERVICE HÉBERGÉ

Candidat exact à nommer.
Même dataset et même révision.
Comparer les capacités requises et les conditions de traitement.

GÉNÉRATION LOCALE

Option à étudier.
Matériel et latence à mesurer.
Données locales ; exploitation à assurer.

Aucun service n'est déclaré meilleur sans résultats comparables.

Une génération hébergée, un RAG local

Décision d'architecture déjà documentée dans le projet.

GÉNÉRER

Service OpenAI-compatible



Narration, choix et décisions

Modèle piloté par configuration.

RETROUVER

JSON de lore → fragments



Embeddings Ollama → Chroma



Contexte filtré par aventure.

ORCHESTRER

LangGraph



Transitions explicites



Outils et moteur de jeu.

Localiser les embeddings ne signifie pas que le contexte reste entièrement local.

Rendre la configuration explicite

Valeurs par défaut du code ; l'environnement peut les remplacer.

GÉNÉRATION

Modèle : OPENAI_MODEL.
Clé dans l'environnement.
Timeout d'appel : 30 s.
Métadonnées sans clé secrète.

RETRIEVAL

RAG_ENABLED configurable.
Ollama : hôte local par défaut.
Embeddings : mxbai-embed-large.
Timeout : 10 s ; 3 tentatives.

FRONTIÈRE API

Choix : 1 000 caractères max.
Quota par défaut : 20 tours par utilisateur et par heure.
Validation de configuration.

Les paramètres décrivent le déploiement ; les secrets restent hors des supports.

Préparer le contexte avant la génération

Vérification locale du corpus : 57 fragments préparés.

CORPUS

6 fichiers de personnages.
3 fichiers de lieux.
57 fragments au dry-run.
Métadonnées de source.

À L'EXÉCUTION

Requête contextualisée.
Recherche filtrée par aventure.
Passages injectés au prompt.
Cache selon le service.

EN CAS D'ÉCHEC

Contexte vide et poursuite du jeu.
Échec visible dans les métriques.
Qualité narrative à surveiller.

Le dry-run vérifie la préparation ; il ne mesure ni les embeddings ni le rappel du RAG.

Maîtriser coûts, données et dépendances

Des arbitrages à chiffrer avant une sélection définitive.

COÛT

Tours × appels par tour × tokens
moyens × tarifs.
Ajouter hébergement, stockage et
calcul local.

DONNÉES

Décrire ce qui entre dans les
prompts. Vérifier les conditions du
fournisseur et les règles de
conservation.

SOBRIÉTÉ

Limiter le contexte utile.
Réutiliser les embeddings.
Comparer qualité et ressources.
Bilan environnemental non
mesuré.

Le registre des coûts et l'analyse fournisseur restent à compléter avec des données datées.

Une solution configurée, une sélection à étayer

Passer de l'architecture à la preuve de pertinence.

C6

Relier le dossier de veille.
Ajouter synthèses datées et
décisions réellement prises.

C7

Besoins et protocole présents.
Comparer au moins deux
candidats.
Compléter budget, données et
critères environnementaux.

C8

Configuration et RAG préparés.
Prouver la connectivité réelle et la
génération dans un environnement
de démonstration.

Transition E3 : exposer, intégrer, tester et surveiller le composant.

Mettre le composant d'IA en service

Call To Adventure · Bloc 2 · E3 : 20 minutes

EXPOSER / INTÉGRER

API authentifiée, connexion au moteur du jeu et gestion des erreurs.

SURVEILLER / TESTER

Métriques techniques, qualité des sorties et vérifications reproductibles.

LIVRER

Workflow de validation, image Docker et procédure de promotion / retour arrière.

Le composant livré comprend le code, les prompts, la configuration et les preuves.

Exposer trois points d'entrée documentés

Contrat versionné dans docs/block2/openapi.yaml.

ÉTAT DE CONFIGURATION

GET /api/v1/ai/health/
Public.
Indique la configuration.
Ne prouve pas un appel réussi au fournisseur.

MÉTADONNÉES

GET /api/v1/ai/configuration/
Authentifié.
Modèles et limites utiles.
Aucune clé ni prompt.

TOUR DE JEU

POST /api/v1/games/
{game_id}/turns/
Session + CSRF.
Avance une sauvegarde possédée.

Une configuration valide ne constitue pas une mesure de disponibilité du modèle.

Contrôler avant de solliciter le modèle

La frontière v1 associe identité, propriété et limites.

IDENTITÉ / PROPRIÉTÉ

Session Django.
Recherche par ID + utilisateur.
Partie étrangère ou absente :
même réponse 404.

REQUÊTE

CSRF requis pour le POST.
Taille et longueur bornées.
Choix comparé à la liste serveur
lorsqu'elle est non vide.

QUOTA

Compteur utilisateur en cache.
Dépassement : HTTP 429.
Un cache partagé et sa
concurrence sont à revoir pour
plusieurs instances.

Les tests prouvent des contrôles précis ; ils ne constituent pas un audit complet de sécurité.

Relier l'interface au moteur narratif

Deux surfaces HTTP réutilisent la logique de jeu.

NAVIGATEUR

play.html → api_step

↓

StepGameView

Parcours actif du jeu.

API DE CERTIFICATION

/api/v1/games/{id}/turns/

↓

Contrôles puis StepGameView

Surface v1 distincte.

MOTEUR COMMUN

GameEngine → LangGraph

↓

RAG, modèle et outils

↓

Récit / combat / fin / erreur.

Le navigateur n'appelle pas directement l'endpoint v1 dans le code consulté.

Valider la forme et protéger la progression

Les appels au modèle peuvent échouer ou produire une réponse inutilisable.

SORTIES TYPÉES

ChoiceOutput : 3 choix distincts,
chacun ≤ 6 mots.
Objectifs et progression : schémas
dédiés.

REPRISES

Retry sur erreurs transitoires.
Délais et tentatives bornés.
Après épuisement : erreur
contrôlée du service.

ÉTAT DU JEU

Une génération échouée ne doit
pas avancer l'état.
Les règles de soin, dégâts et
progression restent côté serveur.

Réussite HTTP, validité du schéma et pertinence narrative sont trois vérifications différentes.

Observer disponibilité, qualité et coût

Collecteurs → Prometheus → Grafana ; règles vers Alertmanager.

SERVICE

Nombre d'appels et erreurs.
Histogramme des durées.
Objectifs : 99 % de succès et p95
< 30 s.

QUALITÉ / RAG

Sorties structurées valides ou
invalides.
RAG : résultats, vides, erreurs et
durées de récupération.

CONSOMMATION

Tokens d'entrée / sortie.
Coût estimé avec tarifs configurés.
Labels sans prompts, identités ni
sessions.

Les SLO sont des objectifs ; aucun historique de production n'est présenté ici.

Transformer un signal en action

Règles présentes ; livraison d'une notification externe à démontrer.

DÉTECTER

3 indisponibilités en 10 min.
Latence élevée persistante.
Sortie structurée invalide.
Erreurs de retrieval.

DIAGNOSTIQUER

Identifier modèle et révision.
Comparer erreurs, durée et RAG.
Reproduire avec un cas minimal
sans donnée personnelle.

AMÉLIORER

Corriger prompt ou configuration.
Rejouer les cas de référence.
Comparer avant / après.
Garder un retour arrière.

Alertmanager utilise un récepteur local : aucune notification externe n'a été prouvée.

Distinguer tests logiciels et évaluation IA

Trois niveaux complémentaires, avec des limites explicites.

INTÉGRATION SIMULÉE

Tests Django et mocks.
Authentification, CSRF, quota,
erreurs, prompts, outils et
transitions.

DATASET HORS LIGNE

4 cas déclarés : choix EN, choix
FR, source RAG et entrée adverse.
Validation du format du dataset.

ÉVALUATION RÉELLE

--live appelle les 2 cas de choix.
Les cas RAG et frontière API
restent marqués hors ligne.
Qualité sémantique non notée.

Le runner actuel ne mesure pas un taux global de qualité du modèle.

33 tests ciblés réussis

Vérifications du 14 septembre 2026 · appels IA simulés

33 TESTS

API et configuration.
Prompts FR, scope / cache RAG.
Soin, objectifs et métriques.
Django : aucun problème détecté.

57 FRAGMENTS

Préparation du corpus réussie.
Aucun calcul d'embedding pendant le dry-run.
Aucune mesure de rappel.

4 CAS DÉCLARÉS

Rapport offline-validation.
Pas de benchmark fournisseur.
Pas de score FR / EN.
Pas de latence modèle mesurée.

Ces résultats démontrent un socle logiciel ; ils ne remplacent pas l'évaluation en conditions réelles.

Automatiser les contrôles avant livraison

Workflow Block 2 AI component versionné dans GitHub Actions.

QUALITÉ

Dépendances verrouillées.
Lint et checks Django.
Tests + seuil de couverture 55 %.
RAG dry-run + dataset offline.

ARTEFACT

Validation des configurations de monitoring.
Build Docker tagué SHA Git.
Smoke test HTTP /health.

ÉVALUATION LIVE

Déclenchement manuel.
Environnement ai-evaluation.
Clé protégée.
Rapport conservé comme artefact.

Le workflow est présent ; aucun succès GitHub distant n'a été vérifié pour ces slides.

Promouvoir un artefact, prévoir le retour arrière

Procédure documentée ; déploiement réel encore à prouver.

PROMOTION

Revoir les tests et le rapport IA.
Publier l'image exacte.
Déployer son digest en staging.
Vérifier santé et parcours.

TRAÇABILITÉ

Révision Git et dataset.
Modèle et paramètres.
Prompts versionnés.
Collection RAG identifiée.
Conserver les rapports.

RETOUR ARRIÈRE

Redéployer le digest précédent.
Restaurer sa configuration.
Revenir à la collection RAG prévue.
Rejouer santé et smoke tests.

Un digest fixe le conteneur ; il ne fige pas à lui seul le comportement d'un fournisseur distant.

Montrer une preuve à chaque frontière

Séquence proposée sur un environnement de démonstration préparé.

API / INTÉGRATION

Montrer configuration sans secret.
Exercer un tour autorisé.
Afficher un refus de propriété.
Expliquer les routes navigateur / v1.

TESTS / SURVEILLANCE

Afficher le lot de tests et le rapport offline.
En environnement prêt : métriques, erreur simulée et alerte.

LIVRAISON

Ouvrir le workflow.
Relier code et artefact.
Présenter un déploiement et un rollback seulement s'ils ont été exécutés.

Restent à prouver : qualité réelle, accessibilité, alerte reçue et livraison effective.

Écarts repérés entre contrat et code

Points identifiés à la lecture ; aucune modification applicative dans cette livraison.

OPENAPI

La liste security encode session OU CSRF, alors que la vue exige les deux.
Réunir les exigences dans le contrat.

ENTRÉES

Le contrat interdit les champs supplémentaires ; la vue ne les rejette pas explicitement.
Un JSON non objet reste à tester.

CHOIX

La comparaison est conditionnée à une liste serveur non vide.
Définir et tester le comportement quand elle est vide.

Conserver ces réserves dans l'argumentaire de sécurité.

Retrouver les preuves C6 à C13

Sources relatives à la racine du dépôt.

E2

C6 : dossier de veille à joindre.

C7 :
ai-requirements-and-selection.md

C8 : runtime_config.py et
documentation RAG.

E3 · SERVICE

C9 : ai_api.py + test_ai_api.py

C10 : game_engine.py et play.html

C11 : observability/ et monitoring/

E3 · QUALITÉ

C12 : evaluation/ et tests Django

C13 : block2-ai.yml

Rapport :
preuves/evaluation-offline.json

La documentation de synthèse se trouve sous docs/block2/.

Défendre les limites du dispositif

Préparer une réponse courte, puis une preuve.

POURQUOI CE MODÈLE ?

Architecture cohérente avec le besoin ; sélection finale à soutenir par un comparatif réel.

COMMENT MESURER ?

Format par schéma ; pertinence par cas et revue humaine ; disponibilité et délai par métriques.

QU'EST-CE QUI MANQUE ?

Veille jointe, benchmark, notation RAG / adversarial, audit accessible, alerte reçue et déploiement tracé.

Un modèle préentraîné déplace les contrôles vers les données, les prompts et l'intégration.